ZOO ANIMAL CLASSIFIER

Documentation and Project by Jomana Shaltout

Contents

Overview

The Zoo Animal Classifier is a simple AI Classification program that classifies animals at a zoo into seven different classes (Mammals, Birds, Reptiles, Fish, Amphibians, Bugs, and Invertebrates).

The Dataset

The dataset consists of 101 animals total and 16 variables that describe the traits of each animal [1].

The dataset contains two .csv files: the first file is 'zoo.csv' which contains the features of the animals and the 'class.csv' is the second file, and it contains the classes the animals are split into (more logically, classified into).

Below is a table that shows the features of both .csv files:

Table 1: Description of Dataset Features.

zoo.csv		class.csv	
Feature	**Description**	**Feature**	**Description**
animal_name	The name of the animal.	Class_Number	The number of the class.
hair	Whether the animal has hair or not.		
feathers	Whether the animal has feathers or not.		
eggs	Whether the animal lays eggs or not.		
milk	Whether the animal has milk (mammary glands) or not.	Number_Of_Animal_Species_In_Class	The number of animals in the class.
airborne	Whether the animal is airborne or not.		
aquatic	Whether the animal is aquatic or not.		
predator	Whether the animal is a predator or not.		

toothed	Whether the animal is toothed or not.	Class_Type	The animal class (e.g. mammal, bird, etc.)
backbone	Whether the animal has a backbone or not.		
breathes	Whether the animal breathes or not.		
venomous	Whether the animal is venomous or not.		
fins	Whether the animal has fins or not.	Animal_Names	A list of the animals that fit within the class.
legs	Whether the animal has legs or not, and how many legs the animal has.		
tail	Whether the animal has a tail or not.		
domestic	Whether the animal is domestic or not.		
catsize	Not clear.		
class_type	The class the animal is in.		

Project Plan and Research

This project is supposed to take the dataset and load it, explore it, split it into two different sets (training and testing), scale the sets, train using KNN algorithm, and evaluate the predictions.

First, we will discuss scaling. Data scaling is the changing of numeric features so that they are on a similar range. This is done because in the KNN algorithm, the model decides based on the distance between points, and a feature with large numbers can unfairly dominate the distance calculation. Scaling is done using a module called StandardScaler.

After scaling, the KNN algorithm trains the model. KNN stands for K-Nearest Neighbours, which is a simple supervised learning algorithm used for classification and regression (in this project it was used for classification only). It looks at the closest training examples to a new data point and assigns the class that appears most (this is called a majority vote) among the neighbours.

Then, the model starts predicting the classes and the predictions are evaluated to determine whether the model performed good or not (by seeing if the test set was classified correctly or not).

Libraries and Modules

Some libraries and certain modules were used in the making of this project:

Libraries

Three libraries were mainly used, and they were Pandas, Matplotlib, and Scikit-Learn.

Pandas was used for the dataset loading and exploring with methods like .describe(), .info, .shape, and others.

Matplotlib was used for plotting some diagrams to visualise results of the evaluation and to display the distribution of the classes.

Scikit-Learn was used for its modules which are: StandardScaler, KNeighborsClassifier, train_test_split, confusion_matrix, and f1_score.

Modules

In Scikit-Learn, there were five modules used.

StandardScaler was used to scale the features where each column has a mean of 0 and a standard deviation of 1. It computes the statistics from the training set, then uses them to transform both the training and test sets.

KNeighborsClassifier classifies a new sample by looking at the 'k' closest training sample and uses a majority vote to choose the final label/class.

Train_test_split was used to divide the dataset into a training set and a testing set where the training set is 80% of the original dataset and the testing set is 20% of the original dataset. The choice is shuffled and not chosen in order. This allows the model to train on one part of the data then evaluates it on the unseen part.

confusion_matrix was used to see how well the model had performed by comparing the true labels with the predicted labels and showing how many were correctly and incorrectly classified.

f1_score is a classification metric that combines both precision (how often is the model's prediction correct?) and recall (out of all real cases, how many did the model find?) into one number.

Plots

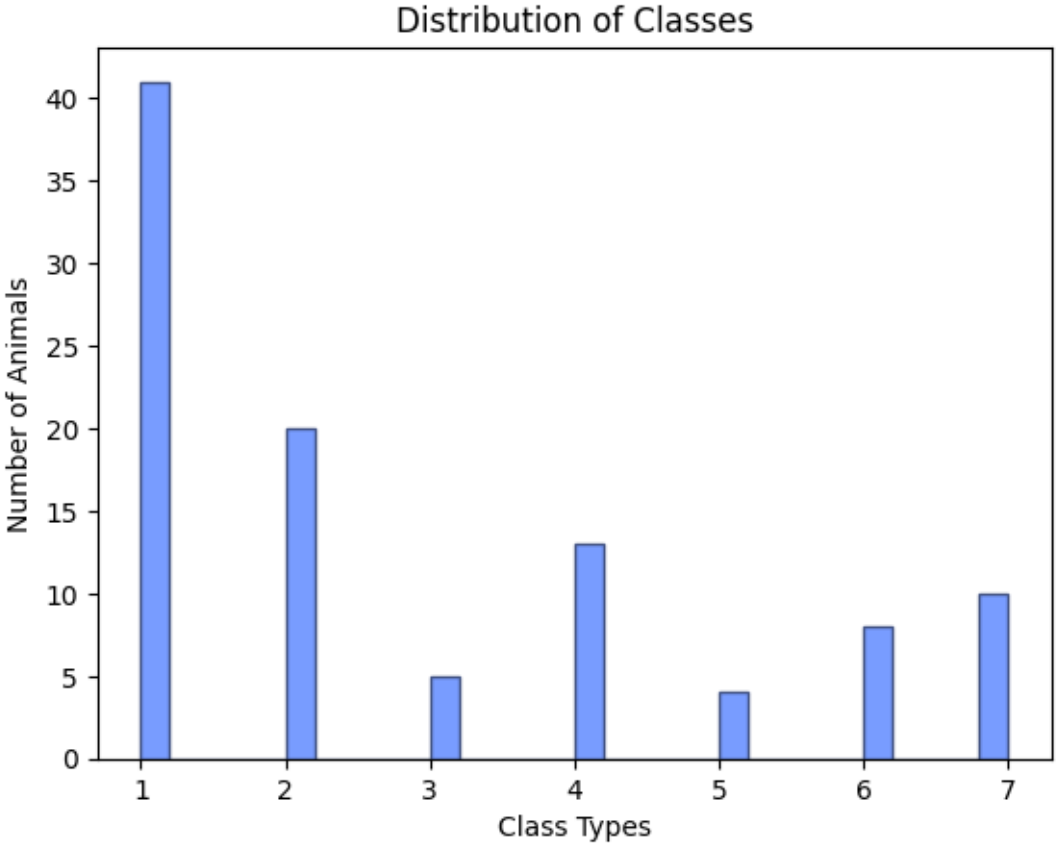

Figure 1: Class Distribution (Number of animals in each class)

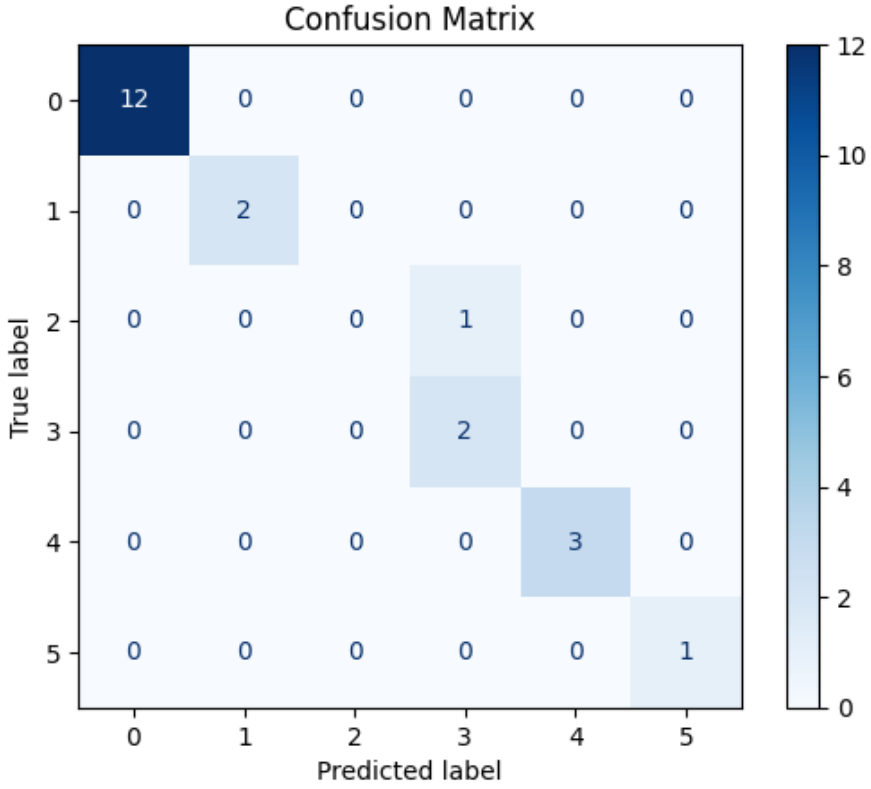

Figure 2: Confusion Matrix

Insight

The Zoo Animal Classifier was determined to be able to efficiently classify zoo animals with only 1 class misclassified out of the rest.

References

[1] Zoo Animal Classification Dataset = https://www.kaggle.com/datasets/uciml/zoo-animal-classification/data s